Programação Orientada a Objetos

Construtores, Destrutores e Separação de Arquivos

Professor: Pedro Montoaneli
pedromontoaneli@teiacoltec.org

Sumário

- Construtores de objetos
  - Definição
  - O ponteiro this
  - Destrutores
  - Exemplo
- Separação de Arquivos
  - Arquivos de cabeçalho
  - Separando interface e implementação
- Exercícios

Definição

- Um construtor é uma função membro que inicializa o objeto de uma classe, podendo ou não receber parâmetros.
- Eles devem ter o **mesmo** nome da classe e **não** devem ter o tipo especificado.
- Toda classe possui um construtor-padrão, fornecido pelo próprio compilador.
- Em uma mesma classe, é possível definir múltiplos construtores.

O ponteiro `this`

- Em qualquer método, o this-> é um ponteiro para o próprio objeto.
- Seu uso é opcional, mas ele se torna obrigatório quando o parâmetro tem o mesmo nome do atributo, que é o jeito que muitos livros e tutoriais escrevem setters.

```
void setNome(string nome) {
    // sem this, nome = nome nao faz nada util
    this->nome = nome;
}
```

Destrutores

- São funções membro especiais chamadas implicitamente na hora de destruir um objeto.
- Destrutores não recebem parâmetros nem especificam tipo de retorno. Além disso, devem sempre ter o mesmo nome da classe.

```
~nomeClasse () {
 //Lógica interna
}
```

- Em geral, são utilizados em objetos que realizam alocações dinâmicas de memória.

Exemplo: Classe “Disciplina” com construtor

```
1.#include <iostream>
2.#include <string>
3.using namespace std;
4.
5.// Definição da Classe
6.class Disciplina {
7.public:
8.
9.    Disciplina(string nome){
10.        setNomeDisciplina(nome);
11.    } // Construtor para a classe
12.
13.    string getNomeDisciplina(){
14.        return nomeDisciplina;
15.    } // Função get
16.
17.    void setNomeDisciplina(string nome){
18.        nomeDisciplina = nome;
19.    } // Função set
20.
21.    void mensagem(){
22.        cout << "Bem vindos à disciplina de " << getNomeDisciplina() << "!" << endl;
23.    }
24.
25.private:
26.    string nomeDisciplina; // Membro de Dados privado
27.}; // Fim da Classe
```

```
28. int main() {
29.     Disciplina circuitos1("Circuitos I");
30.     Disciplina circuitos2("Circuitos II");
31.
32.     cout << "Primeira disciplina: " <<
    circuitos1.getNomeDisciplina() << endl;
33.     cout << "Segunda disciplina: " <<
    circuitos2.getNomeDisciplina() << endl;
34.
35.     return 0;
36. }
```

Arquivos de Cabeçalho

- Para uma classe poder ser aproveitada em outros códigos, geralmente sua definição é feita em um arquivo de cabeçalho, ou arquivo **header**.
- Um arquivo header deve ser salvo com a extensão .h
- No arquivo do código fonte (.cpp), o arquivo header é inserido usando o comando `#include`.

```
1.#include <iostream>
2.#include <string>
3.#include "Disciplina.h"
4.using namespace std;
5.
6.int main() {
7.    Disciplina circuitos1("Circuitos I");
8.    cout << "Primeira disciplina: " << circuitos1.getNomeDisciplina() << endl;
9.    return 0;
10.}
```

Separando interface e implementação

- A interface de uma classe descreve os membros e funções presentes na classe.
- Na interface, as funções são substituídas por **protótipos de funções**.
- Um protótipo de função informa apenas seu nome, tipo de retorno e tipo de parâmetros.

```
1.#include <string>
2.using namespace std;
3.
4.// Definição da Classe
5.class Disciplina {
6.public:
7.
8.    Disciplina(string nome); // Construtor
9.    string getNomeDisciplina(); // Função get
10.    void setNomeDisciplina(string nome); // Função set
11.    void mensagem(); // Função Mensagem
12.
13.private:
14.    string nomeDisciplina; // Membro de Dados privado
15.}; // Fim da Classe
```

Separando interface e implementação

- No arquivo de implementação, as funções são implementadas a partir dos protótipos definidos no arquivo de interface.
- A extensão do arquivo de implementação é .cpp e não deve conter a função `main()`.

```
1.#include <iostream>
2.#include <string>
3.#include "Disciplina.h"
4.using namespace std;
5.
6.Disciplina::Disciplina(string nome){
7.    setNomeDisciplina(nome);
8.} // Construtor para a classe
9.string Disciplina::getNomeDisciplina(){
10.    return nomeDisciplina;
11.} // Função get
12.void Disciplina::setNomeDisciplina(string nome){
13.    nomeDisciplina = nome;
14.} // Função set
15.void Disciplina::mensagem(){
16.    cout << "Bem vindos à disciplina de " << getNomeDisciplina() << "!" << endl;
17.}
```

Exercícios

Exercício 1

Crie uma classe `Aluno` com os atributos privados `nome`, do tipo string, e `nota`, do tipo double. Forneça dois construtores: um sem parâmetros que inicializa o nome como "Sem nome" e a nota como zero, e um com dois parâmetros, nome e nota. Nos dois construtores, use `this->` nos setters. Forneça `getNome`, `setNome`, `getNota` e `setNota`, onde o setter de nota recusa valores fora do intervalo de 0 a 10. Adicione um método `situacao()` que imprime "Aprovado" se a nota for maior ou igual a 6 e "Reprovado" caso contrário. No `main`, crie dois alunos, um com cada construtor, e chame `situacao()` nos dois.

Exercício 2

Pegue a classe `Aluno` do exercício anterior e separe-a em três arquivos: Aluno.h com a definição e os protótipos, Aluno.cpp com a implementação, e main.cpp com a função principal. Compile usando os três passos no terminal e mostre o programa rodando.